

Паттерны проектирования

Компоновщик и Декоратор

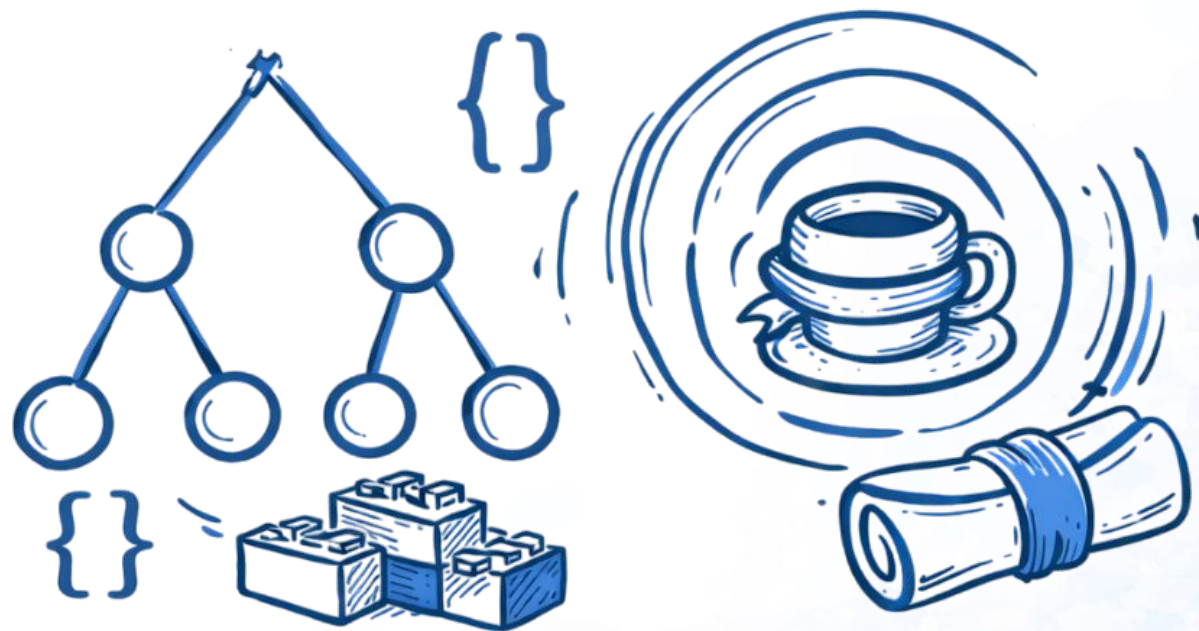
Структурные паттерны для создания гибких объектно-ориентированных систем

Выполнили:

Назарова Елизавета

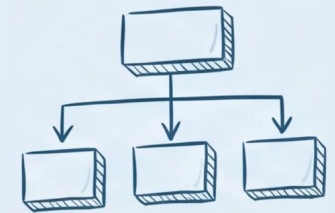
Ксения Фурсова

5030102/40001



Введение в структурные паттерны

«Паттерны проектирования — проверенные типовые решения часто встречающихся проблем в проектировании программного обеспечения»

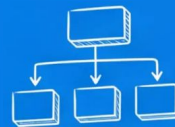


Порождающие

Creational

Управляют созданием
объектов

Singleton, Factory, Builder...



Структурные

Structural

Организуют взаимосвязи
классов и объектов

Компоновщик, Декоратор...



Поведенческие

Behavioral

Описывают взаимодействие
объектов

Strategy, Observer, Command...

В этой презентации мы изучим Компоновщик и Декоратор

Паттерн Компоновщик

Цель

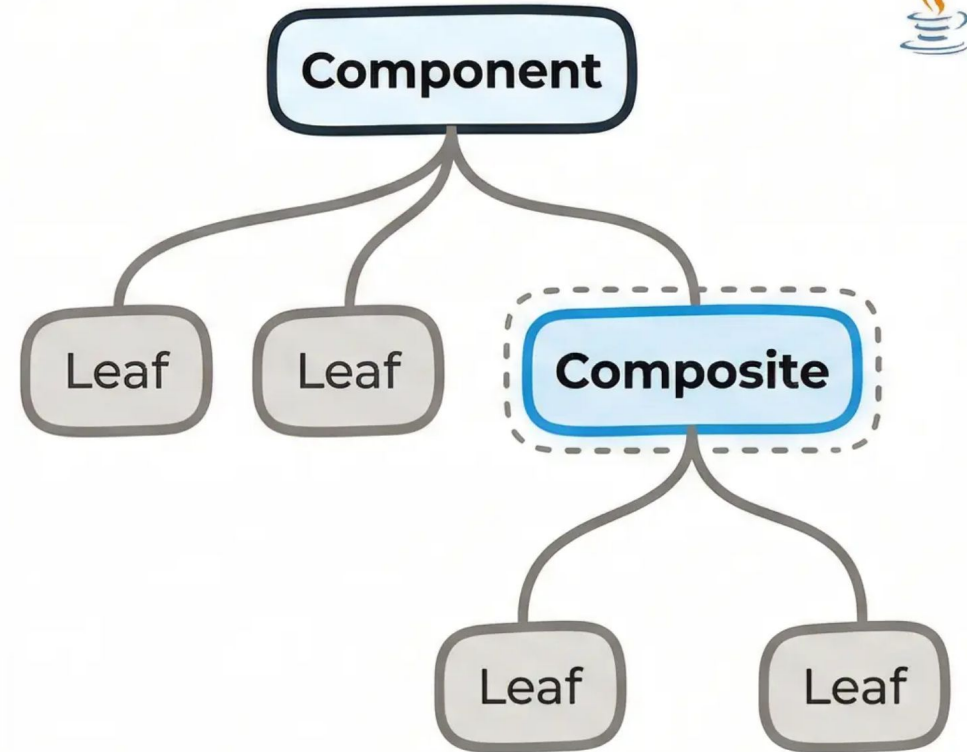
Скомпоновать объекты в структуры по типу "дерева", позволяя клиентам единообразно трактовать отдельные и составные объекты.

Для чего используется:

Для группировки мелких компонентов в более крупные, которые, в свою очередь, могут стать основой для ещё более крупных структур.

Пример использования :

- для представления иерархии "часть-целое";
- мы хотим, чтобы клиенты одним способом трактовали как отдельные, так и составные объекты.



Структура паттерна Компоновщик

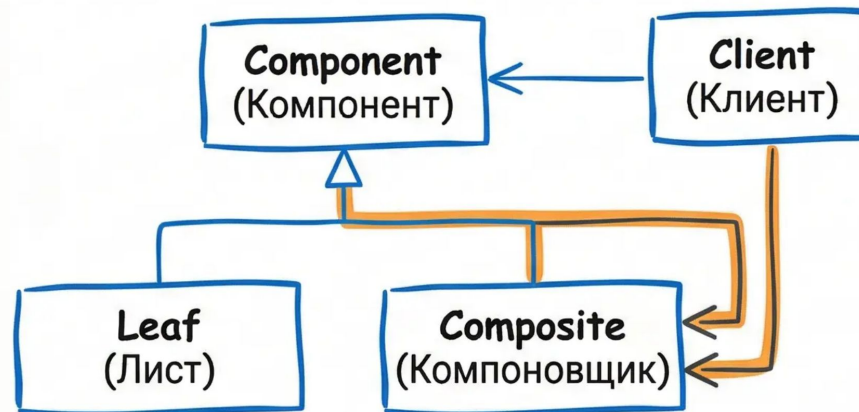
Ключевые участники паттерна и их взаимодействие в иерархической структуре

Component (Компонент)

Общий интерфейс для всех объектов композиции
Определяет операции для листьев и составных объектов
Может содержать методы управления потомками

Leaf (Лист)

Листовые узлы без дочерних элементов
Реализует операции Component
Представляет примитивные объекты композиции

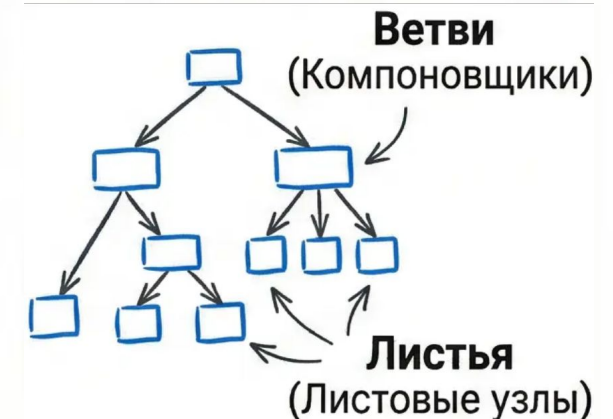


Client (Клиент)

Взаимодействует через интерфейс Component
Не различает листья и составные объекты

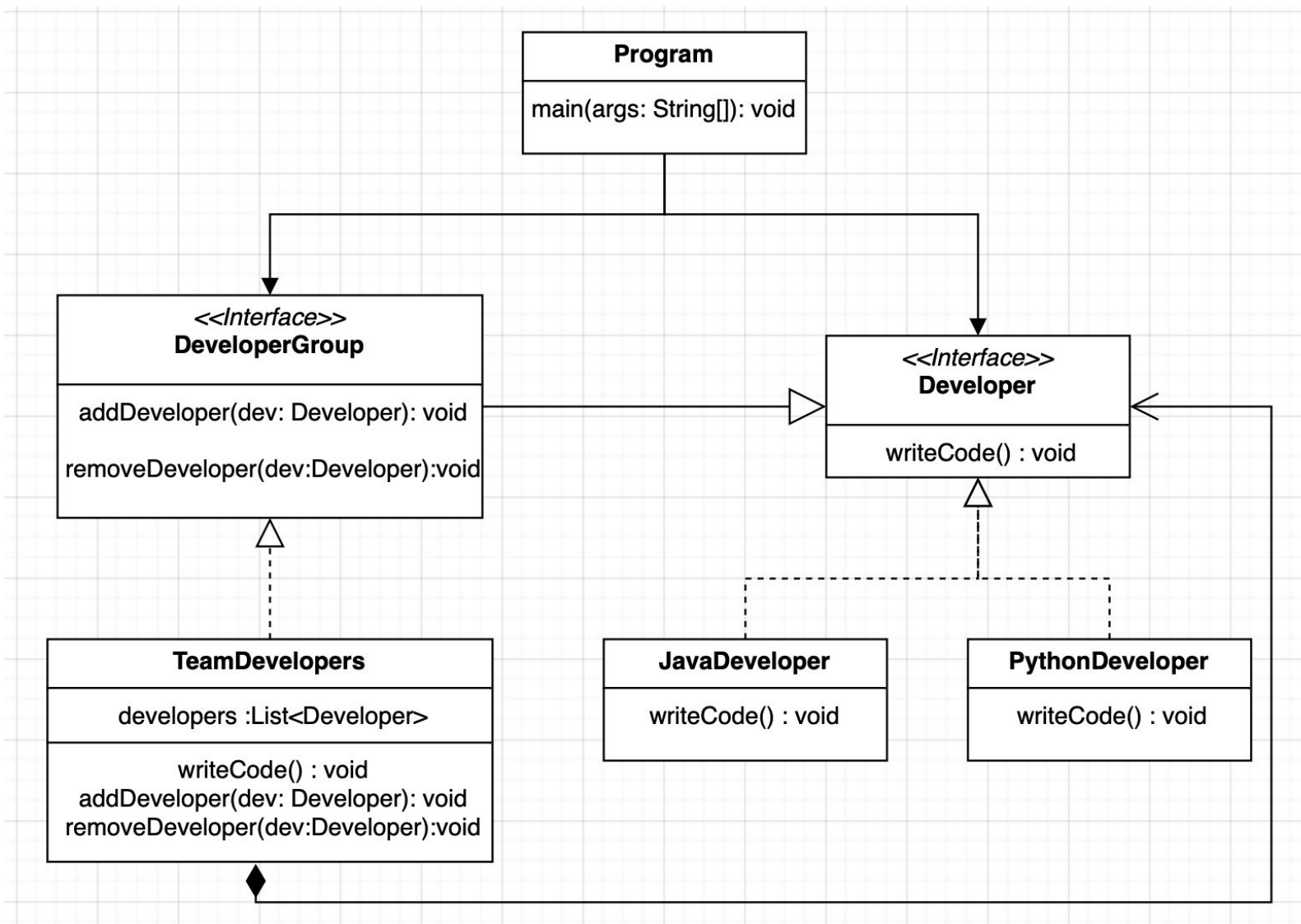
Composite (Компоновщик)

Составные узлы с дочерними компонентами
Хранит ссылки на потомков
Делегирует операции дочерним объектам



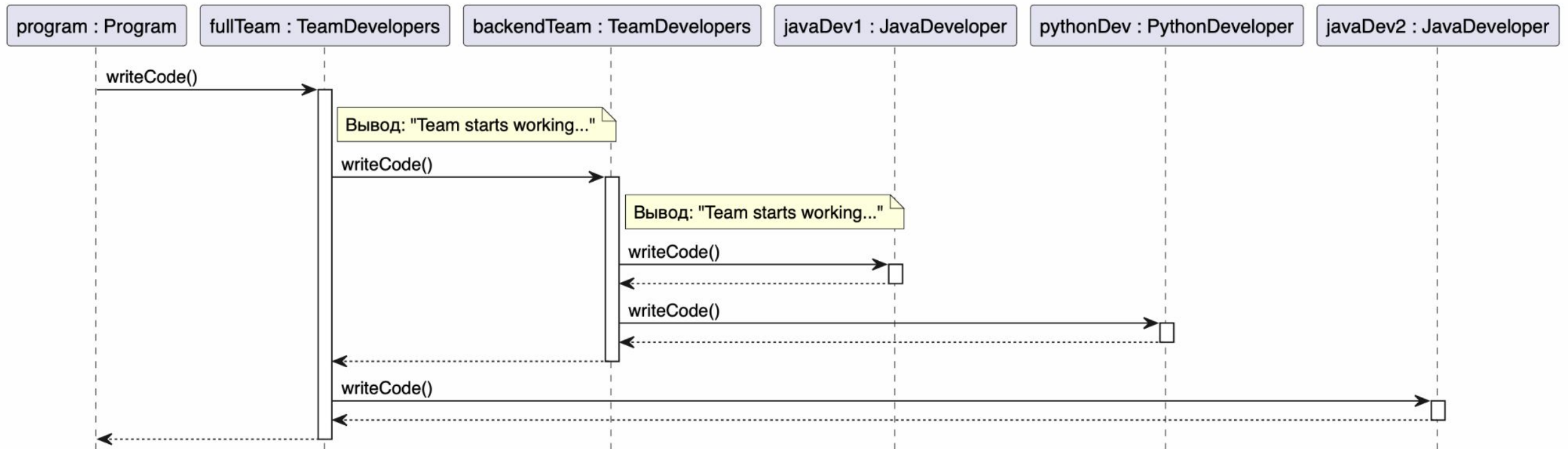
UML: Диаграмма классов — Компоновщи

Структура классов, соответствующая приведённому коду Java



UML: Диаграмма последовательности – Компоновщик

Диаграмма последовательности выполнения writeCode() (Composite)



Преимущества и недостатки Компоновщика

Преимущества

✓ 1. Открытость для расширения (ОСР)

Чтобы добавить нового разработчика - не нужно менять существующий код. В Program просто добавляем:

```
teamDevelopers.addDeveloper(new GoDeveloper());
```

Добавляем новый класс, но текущий код не меняется.

✓ 2. Прозрачность клиентского кода

Клиентский код не меняется - вызывается один и тот же метод.

```
Developer developer = new JavaDeveloper();  
developer.writeCode(); // одиночный объект  
Developer team = new TeamDevelopers();  
team.writeCode(); // группа
```

Клиент работает через общий интерфейс Developer. Ему всё равно - это один разработчик или команда.

Недостатки

✗ 1. Слишком общий интерфейс

```
Developer dev = new JavaDeveloper();  
dev.addDeveloper(new JavaDeveloper());
```

Листовые объекты (JavaDeveloper, PythonDeveloper) будут вынуждены реализовывать методы add/remove, которые им не нужны.

✗ 2. Сложность ограничений на типы

```
team1.addDeveloper(team2); // рекурсия
```

Сейчас TeamDevelopers может содержать любого Developer, включая другие команды.

Трудно запретить определённые комбинации дочерних объектов на уровне типов.

Паттерн Декоратор

Суть

Декоратор позволяет динамически добавлять новую функциональность к объектам, не изменяя их исходный код.

- Динамическое расширение поведения
- “Оборачивание” объектов
- Прозрачность использования
- Комбинируемость
- Следование принципу Open/Closed (O из SOLID)

Паттерн Декоратор

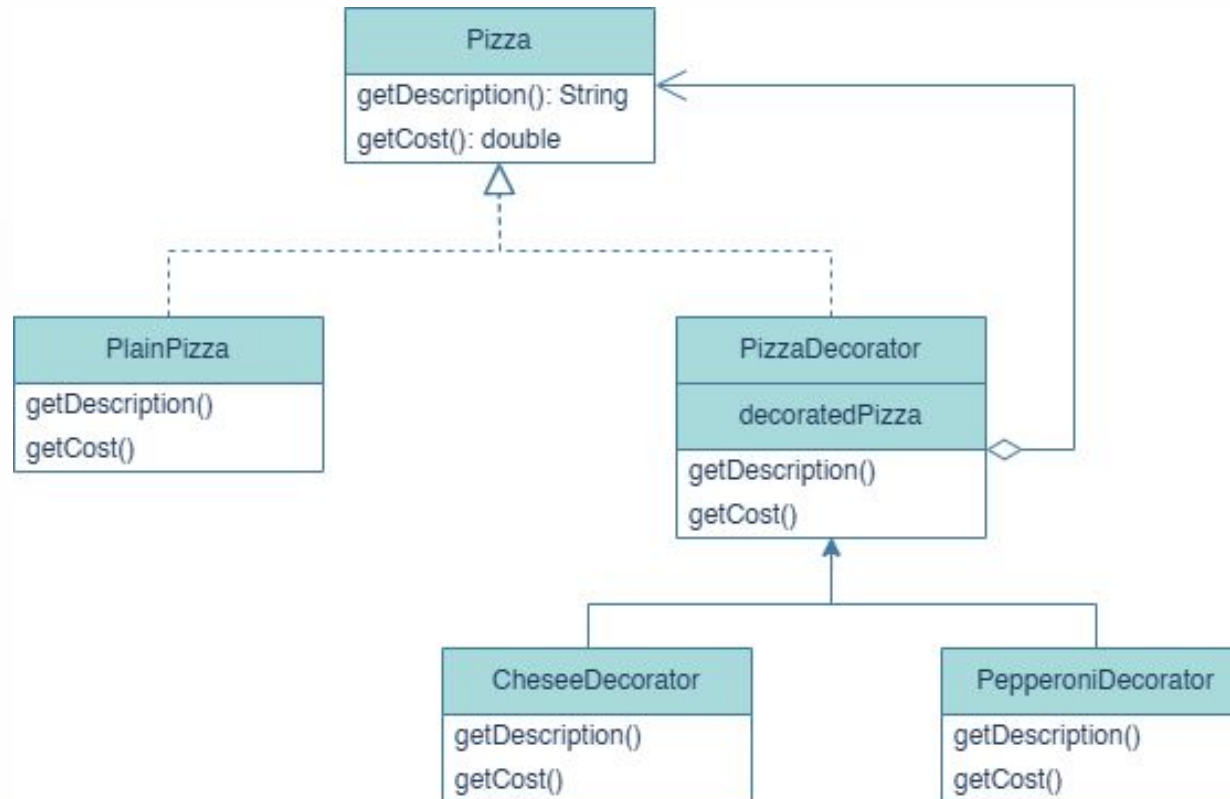
Когда применять?

- ★ нужно добавлять функциональность объекту динамически
- ★ нужно избежать большого количества подклассов
- ★ требуется комбинировать поведение



Структура паттерна Декоратор

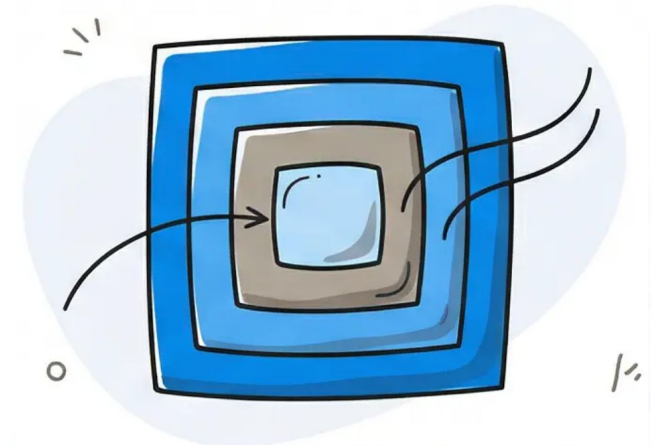
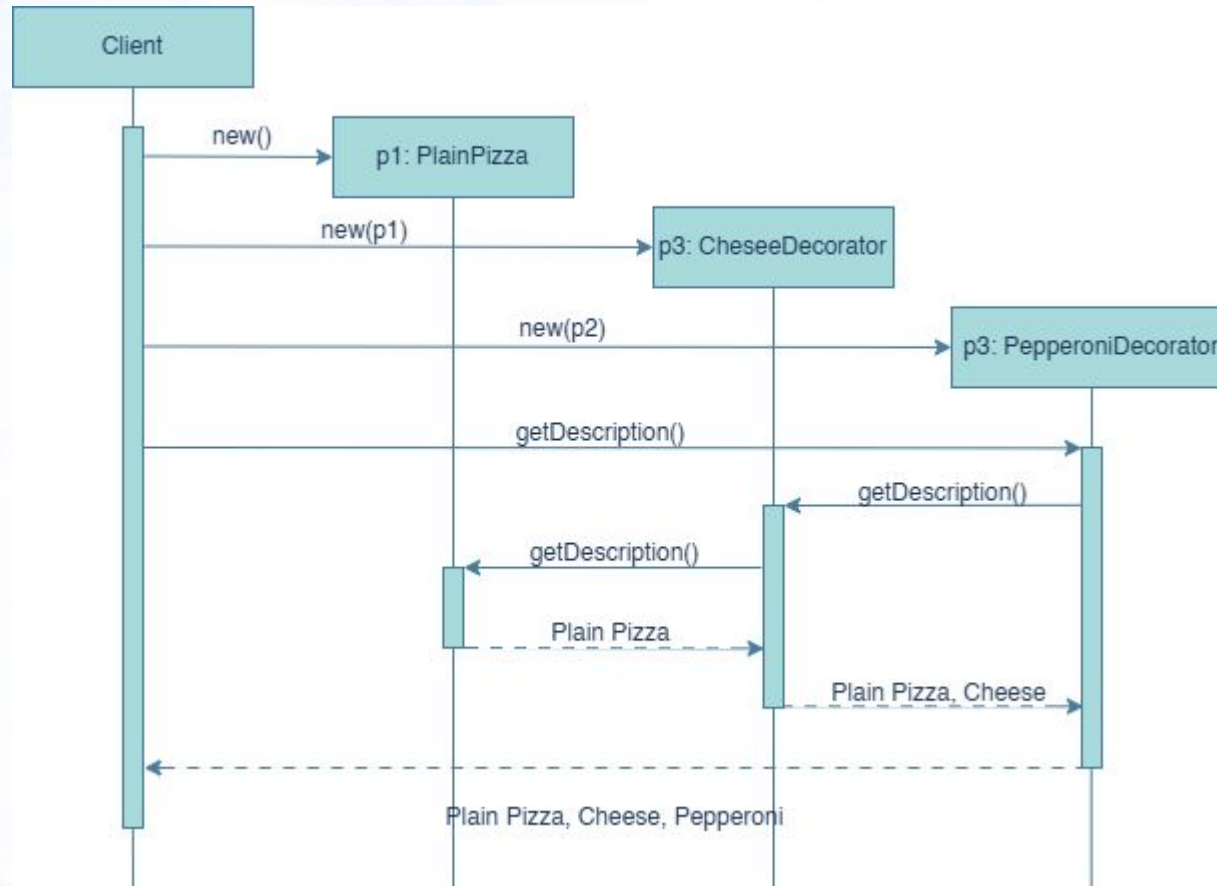
UML диаграмма классов на основе примера кода в Java



Декораторы оборачивают объекты и делегируют им вызовы, добавляя свое поведение до или после вызова

Диаграмма последовательности – Декоратор

На основе примера кода в Java



Преимущества и недостатки Декоратора

Преимущества

Гибкое расширение поведения

```
pizza = new CheeseDecorator(pizza);
```

Альтернатива наследованию

```
public class CheeseDecorator extends  
PizzaDecorator {...}
```

Комбинируемость

```
pizza = new CheeseDecorator(pizza);  
pizza = new PepperoniDecorator(pizza);
```

Следование ОCP (SOLID)

```
public class PlainPizza implements Pizza { ... }
```

Недостатки

Много мелких классов

```
public class PepperoniDecorator extends  
PizzaDecorator { ... }  
public abstract class PizzaDecorator  
implements Pizza { ... }  
public abstract class PizzaDecorator  
implements Pizza { ... }
```

Сложно понимать и отлаживать

Из-за цепочки обёрток неочевидно, что происходит

Зависимость от порядка декораторов

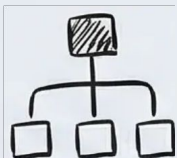
Разный порядок может давать разный результат

Сравнительная таблица: Компоновщик и Декоратор

Сравнение по ключевым критериям

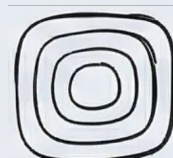
Критерий	Компоновщик (Composite)	Декоратор (Decorator)
Назначение	Организация объектов в дерево для единообразного работы с отдельными и составными элементами	Динамическое добавление поведения объекту через цепочку оберток без изменения его класса
Структура	Дерево объектов: листья и составные узлы реализуют общий интерфейс	Цепочка оберток: декораторы наследуют тот же интерфейс и делегируют вызовы
Гибкость	Статическая — структура формируется при создании объектов	Динамическая — поведение добавляется и удаляется во время выполнения
Решаемая проблема	Устранение различий между работой с одним объектом и группой объектов (часть-целое)	Избежание взрыва подклассов при необходимости многочисленных комбинаций поведений
Примеры использования	UI-компоненты (кнопки и панели), файловые системы (файлы и папки)	Java I/O Streams, логирование, шифрование, сжатие данных

Заключение



Компоновщик

Унифицирует работу с иерархическими структурами — клиент взаимодействует с отдельными элементами и группами через единый интерфейс



Декоратор

Динамически расширяет функциональность объектов без изменения их базовой структуры — гибкая альтернатива наследованию

Оба паттерна демонстрируют силу абстракции и композиции — ключевых принципов создания гибких, масштабируемых и легко поддерживаемых ООП-систем на Java

Спасибо за внимание!